A. Trace rules

You write	Constructor line(s)	Destructor line(s)
A a;	default	at its }
A b = a; A b(a);	copy	at its }
b = a;	none: assignment	none
f(a) by value	1 copy at the call	when f returns
f(a) by reference	none	none
return input; (param)	copy, never elided	at f's end
A y = f(a);	return value **is** y	y at its }
f(a); unused result	param, return	temp at ;, param
return A(…); return local;	elided: 0 copies	none extra
derived object	base, members in order, body	reversed
: public C, public D	C, D (order after colon)	D, C
virtual base	once, first, by most-derived	once, last
Base* p = new Derived; delete p;	Base, Derived	virtual: Derived, Base; else Base only
Base b = derived; f(Base b) with a Derived	sliced: Base copy	Base only

Q20 A foo(A input) { cout << "Look ma, I'm fooing!" << endl; return input; }

A littleA;	1 Default constructor!
foo(littleA);	2 Copy constructor! (input) 3 Look ma, I'm fooing! 4 Copy constructor! (return) 5 Destructor! (temporary) 6 Destructor! (input)
A anotherA = littleA;	7 Copy constructor!
A yetAnotherA = foo(littleA);	8 Copy constructor! (input) 9 Look ma, I'm fooing! 10 Copy constructor! (yetAnotherA) 11 Destructor! (input)
end of main	12 Destructor! (yetAnotherA) 13 Destructor! (anotherA) 14 Destructor! (littleA)

Tally: 1 default, 5 copy, 2 fooing, 6 destructor.

ParentChild Child c; passByValue(c); passByReference(c); 1 Parent default constructor. 2 Child default constructor. 3 A Parent copy was made. 4 in passByValue 5 Parent destructor. 6 in passByReference 7 Child destructor. 8 Parent destructor.

Q27 E : public C, public D; C, D from B; B from A. **Q28**: B : virtual public A, C : virtual public B, D still public B.

Q27 (14)	Q28 (12)
A allocated	A allocated
B allocated	B allocated
C allocated	C allocated
A allocated	B allocated
B allocated	D allocated
D allocated	E allocated
E allocated	E deallocated
E deallocated	D deallocated
D deallocated	B deallocated
B deallocated	C deallocated
A deallocated	B deallocated
C deallocated	A deallocated
B deallocated	
A deallocated	

Why: virtual base A is built once, first, by E; C's B is virtual, D's B is not, so B prints twice and A once.

B. Pointers, references, memory

Pointer: a box holding an address. Reference: a second name for a box.

```
int n = 5; int* p = &n; *p = 7; // n is 7
int& r = n; r = 9;  // n is 9
r = m;  // copies m's value into n
```

string* success = &s; *success is the string; &success is the pointer's address, a string**.

```
void swapV(int a, int b);  // no effect
void swapP(int* a, int* b);
void swapR(int& a, int& b);
swapV(a, b); swapP(&a, &b);
swapR(a, b);
```

const T&: no copy, no change. Never return a reference to a local. Leak: new without delete. Dangling: used after delete.

```
int x = 5;  // static: stack, freed at }
int* q = new int(5);  // heap, until delete
int* arr = new int[n];  // run-time size
delete q; q = nullptr; delete[] arr;
pt->x;  // same as (*pt).x
```

v.at(5) throws out_of_range; v[5] is unchecked.

C. Definitions in three moves

Q1 Copy ctor (4). Builds a new object from an existing one: Fraction(const Fraction& other) (by value would recurse). Runs on T b(a); T b = a; pass/return by value; not b = a; Default is shallow.

Q2 Reference vs pointer (3). Reference: an alias, another name for a variable; initialized at once, never re-seated: int& r = n; Pointer: holds an address; may be null or re-pointed: int* p = &n;

Q3 Pass by (4). Value copies (caller safe). Pointer passes the address (changes the pointee; may be nullptr). Reference: no copy, changes the caller's variable; const T& for big objects.

Q4 (4). *success: the string, to read or change it. &success: the pointer's own address (string**), so a function can re-point it: void reset(string** pp).

Q5 Static vs dynamic (4). Static: int x = 5; stack, compile-time size, freed at }. Dynamic: int* p = new int[n]; heap, run-time size, until delete[] p;

Q6 Leak (4). new memory never deleted: unreachable until exit. Prevent: pair new/delete; free in the destructor. void leak() { int* p = new int[100]; delete[] p; }

Q7 const (6). a. void print(const Fraction& f): will not modify the argument (with &: no copy). b. int getNumerator() const; changes no member; callable on a const object.

Q8 Destructor (3) at end of scope (and unwinding), on delete, a temporary at its ;, members and bases after the body, statics at exit. Reverse order.

Q9 Init (4). int x = 0; int x(0); int x{0}; Prefer braces: no narrowing (int y{3.7}; won't compile; others truncate).

Q10 (2). :: scope resolution: std::cout, Fraction::Fraction(…) outside the class. . a member of an object; -> through a pointer.

Q11 class vs struct (2). Only default access: struct public, class private (members, inheritance).

Q12 Forward declaration (3). class Car; before the definition: Car*, Car& usable (mutual references, fewer includes); Car c; is incomplete.

Q13 Abstract (3). At least one pure virtual: virtual double area() const = 0; No keyword; cannot instantiate; derived stays abstract until all are overridden.

Q14 Interface (2). No keyword: a pure abstract class (only pure virtuals, a virtual destructor), inherited.

Q15 Diamond (6). a. student, mathematician inherit person; math_student both: two persons unless virtual. b. bob.all_info() ambiguous: bob.mathematician::all_info() or override. c. Most-derived calls person(name); middle calls ignored.

Q16 Copy-and-swap (3). T& operator=(T other): 1. by-value copy 2. mySwap every member 3. other dies with the old data. Errors: a T& parameter; std::swap on the whole object (recurses).

D. Right-left rule

1. Start from variable name (var)
2. Keep going right. Read what we see until we see right parentheses)
3. Then go left of var. Read what we see until we see left parentheses (
4. Exit parentheses and go right. Read what we see until right parentheses or semicolon
5. Go left of where we exited bracket. Read what we see until left parentheses or finish

int ***ppp;	pointer to a pointer to an int pointer
int (**ppa)[];	pointer to a pointer to an array of ints
int (**ppf)();	pointer to a pointer to a function returning an int
int *(*pap)[];	pointer to an array of int pointers
int (*paa)[][];	pointer to an array of arrays of ints (does not compile)
int (*paf)[]();	pointer to an array of functions returning an int (does not compile)

int (&ref)[100] = a; reference to an array. swap(int*& p, int*& q) references to pointers. int *(*(*fp1)(int))[10]; pointer to a function (int) returning a pointer to an array of 10 int pointers.

E. Fraction, complete

```
#include <iostream> <fstream>
<sstream> <stack> <vector> <algorithm>
<stdexcept> <string> (one per line); using
namespace std;
class Fraction {
private:
  int numerator;
  int denominator;
public:
  // Q21: initializer list, then the check
  Fraction(int numerator, int denominator)
    : numerator(numerator),
      denominator(denominator) {
    if (denominator == 0)
      throw invalid_argument(
        "Denominator cannot be zero");
  }
  // Q22: const &, else it would recurse
  Fraction(const Fraction& other)
    : numerator(other.numerator),
      denominator(other.denominator) {}
  ~Fraction() {}  // Q23: nothing to free
  int getNumerator() const {
    return numerator; }
  // Q24: write to os, return os
  friend ostream& operator<<(
      ostream& os, const Fraction& f) {
    os << f.numerator << "/"
       << f.denominator;
    return os; }
  // Q25: read into f, return is
  friend istream& operator>>(
      istream& is, Fraction& f) {
    is >> f.numerator >> f.denominator;
    return is; }
  friend Fraction operator+(
      const Fraction& a, const Fraction& b)
  { return Fraction(
      a.numerator * b.denominator
        + b.numerator * a.denominator,
      a.denominator * b.denominator); }
  friend bool operator==(
      const Fraction& a, const Fraction& b)
  { return a.numerator * b.denominator
      == b.numerator * a.denominator; }
  friend bool operator<(
      const Fraction& a, const Fraction& b)
  { return a.numerator * b.denominator
      < b.numerator * a.denominator; }
};
int main() {  // Q26
  ifstream in("Fractions.txt");
  if (!in.is_open()) {
    cerr << "Could not open" << endl;
    return 1; }
  stack<Fraction> st;
  int num, den;
  while (in >> num >> den)  // to EOF
    st.push(Fraction(num, den));
  while (!st.empty()) {  // last in, first out
    cout << st.top() << endl;
    st.pop(); }
  try { Fraction bad(1, 0); }
  catch (const invalid_argument& e) {
    cout << e.what() << endl; }
}
```

Rows 1 2 3 4 5 6 print 5/6 3/4 1/2
Denominator cannot be zero. Or:

```
string line;
while (getline(in, line)) {
  istringstream iss(line);
  iss >> num >> den;
  st.push(Fraction(num, den)); }
```

Vector variant, prints 1/2 3/4 5/6:

```
vector<Fraction> v;
while (in >> num >> den)
  v.push_back(Fraction(num, den));
sort(v.begin(), v.end());  // operator<
for (const auto& f : v) cout << f << " ";
```

Marks: Q24 returns os; Q25 reads into f, returns is; Q26: open check, loop, push, show.

F. Deep copy, copy-and-swap, abstract, interface, diamond

```
class Character {
  int* skillIds;
  int size;
public:
  Character(int n)
    : skillIds(new int[n]{}), size(n) {}
  Character(const Character& o)  // deep
    : skillIds(new int[o.size]),
      size(o.size) {
    for (int i = 0; i < size; ++i)
      skillIds[i] = o.skillIds[i]; }
  ~Character() { delete[] skillIds; }
  friend void mySwap(Character& a,
                     Character& b) {
    swap(a.skillIds, b.skillIds);
    swap(a.size, b.size); }  // all members
  Character& operator=(Character other)
  { // 1. by value: copy ctor made other
    mySwap(*this, other);  // 2. swap all
    return *this; }  // 3. other dies: old
};
```

Errors: Character& other (no copy); std::swap(*this, other) (recurses).

```
class Shape { public:
  virtual double area() const = 0;
  virtual ~Shape() {} };
class Circle : public Shape { double r;
public: Circle(double r) : r(r) {}
  double area() const override {
    return 3.14 * r * r; } };
Shape* s = new Circle(2);
cout << s->area();  // 12.56
delete s;  // ~Circle then ~Shape
// Shape sh;  error: abstract
```

Interface: only pure virtuals + virtual ~Drawable() {}.

```
class person { public: string name;
  person(string n = "?") : name(n) {} };
class student
  : public virtual person { public:
  student(string n) : person(n) {}
  void all_info() { cout << name; } };
class mathematician
  : public virtual person { public:
  mathematician(string n) : person(n) {}
  void all_info() { cout << name; } };
class math_student : public student,
  public mathematician { public:
  math_student(string n) : person(n),
    student(n), mathematician(n) {} };
// most-derived class constructs person;
// the middle classes' person(name)
// calls are ignored
math_student bob("Bob");
// bob.all_info();  error: ambiguous
bob.mathematician::all_info();  // Bob
```

Friend: not mutual, not transitive, not inherited. virtual + Base*/Base& runs the derived version; override is checked; Base b = derived; slices.

G. Exceptions

```
struct MyErr : exception {
  const char* what() const noexcept
    override { return "MyErr"; } };
try { throw invalid_argument("bad"); }
catch (const invalid_argument& e) {
  cerr << e.what(); }  // child first
catch (const exception& e) {
  cerr << e.what(); }  // parent after
catch (...) { throw; }  // last; rethrows
```

By reference (by value slices); first match wins. Uncaught → std::terminate → abort. Unwinding destroys locals in reverse. Broken noexcept → terminate; destructors are noexcept. A throwing constructor makes no object. logic_error (invalid_argument, out_of_range), runtime_error. throw "msg"; is a const char*.

H. STL

```
map<string, int> bag;
bag.insert({"Slime", 1});
bag.insert({"Slime", 5});  // refused: 1
bag["Slime"] = 5;  // overwrites
cout << bag["Gel"];  // 0: [] creates
for (auto it = bag.begin();
     it != bag.end(); ++it)
  cout << it->first << it->second;
for (auto& x : v) x *= 10;  // auto copies
```

set skips equals; priority_queue top is the largest; end() is one past the last.

Kind	Operations	Containers
input	read, ++	istream_iterator
output	write, ++	ostream_iterator
forward	++	forward_list, unordered_map
bidirectional	++ --	list, map, set
random access	it + n it[n] <	vector, deque, array

```
bool byHp(const Mob& a, const Mob& b)
  { return a.hp < b.hp; }
sort(v.begin(), v.end());  // operator<
sort(mobs.begin(), mobs.end(), byHp);
l.sort();  // sort() needs random access
```

partition loses order, stable_partition keeps it. Strict weak ordering: irreflexive, antisymmetric, transitive.

I. Static, copy elision

```
class Mob { public:
  static int alive;  // define outside
  const static int maxHp = 9;  // may
  constexpr static int level = 3;  // must
  Mob() { ++alive; }
  static int count() { return alive; } };
int Mob::alive = 0;  // no "static" here
```

One copy per class. Static function: no this, "can only access static data", not virtual or const.

```
Number createNumber(int n) {
  return Number(n); }  // RVO
Number n = createNumber(5);  // 1 obj
Number n(Number(2));  // 1 object
Number t(n); return t;  // NRVO: optional
Number f(Number p) { return p; }  // 2
copies
```

J. Week 1 syntax

-7 / 2 is -3, -7 % 2 is -1. fixed << setprecision(2): 3.14.

```
cin.clear(); cin.ignore(numeric_limits
  <streamsize>::max(), '\n');  // bad input
```